IMPLEMENTATION 1 – SUMMARY REPORT

Handwritten Digit Recognition System

Team: Sparkling

Name	Registration Number	Index Number
A.S.M Musaffar	ICT/2022/065	5670
M. Ridhurshan	ICT/2022/079	5683
R. Shobana	ICT/2022/016	5623
S. Danusha	ICT/2022/017	5624
A.N.M. Rifky	ICT/2022/127	5728

Faculty of Applied Sciences

Rajarata University of Sri Lanka

2021/2022 Batch

Table of Contents

1. Introduction ……. 3
2. Dataset Preparation……... 3
3. Image Preprocessing……. 5
4. Baseline Model Design……. 5
5. Model Training……. 7
6. Initial Model Evaluation……. 7
7. Sample Predictions…… 8
8. Error Analysis…… 9
9. Overfitting/Underfitting Analysis……. 10
10. Class Imbalance Analysis…… 10
11. Model Saving……. 11
12. Initial Observations…….. 11
13. Conclusion…… 11

1. Introduction

This project implements a handwritten digit recognition system using a Convolutional Neural Network (CNN). The objective of Implementation 1 is to develop a baseline image classification model capable of identifying handwritten digits (0–9) using the MNIST dataset.

The focus of this phase is correctness and structured implementation rather than maximum performance.

1.1 Project Repository

The complete source code, trained model files, and experiment documentation for Implementation 1 are available in the public GitHub repository:

https://github.com/ASMmusaffar/Sparkling-Handwritten-Digits-Recognizer/tree/main

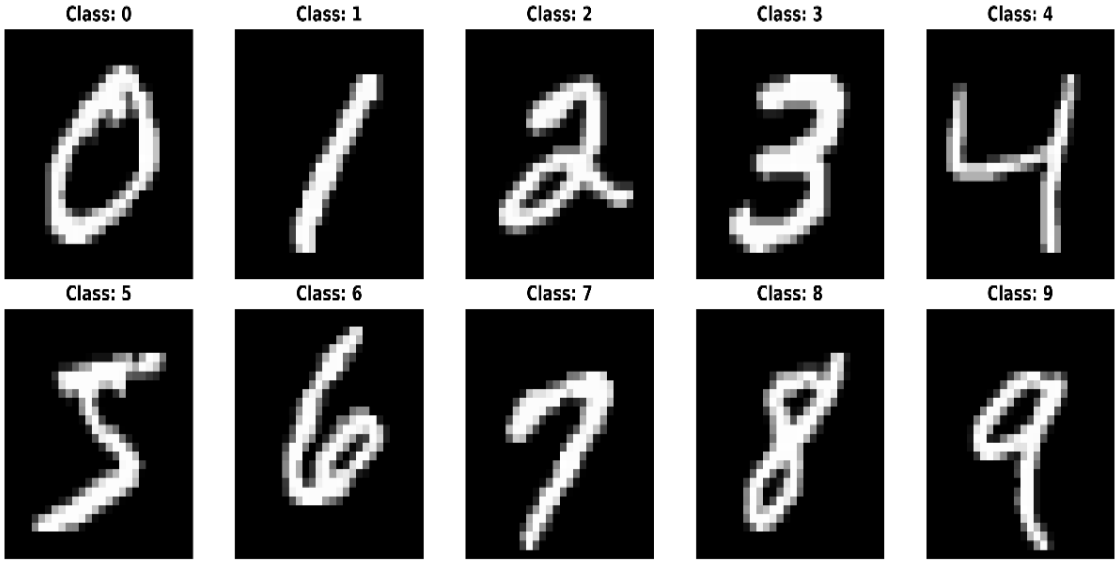
2. Dataset Preparation

2.1 Dataset Source

The dataset used is the **MNIST dataset**, available through Keras.

- Total original training images: 60,000
- Total test images: 10,000
- Image size: 28 × 28 pixels
- Image type: Grayscale
- Number of classes: 10 (Digits 0–9)

FIGURE 1: Sample Handwritten Digits from MNIST Dataset



2.2 Dataset Reduction

For experimentation and computational efficiency, the training dataset was reduced from 60,000 to 6,000 samples.

- Training samples used: 6,000
- Testing samples used: 10,000

This reduction enables faster training while maintaining sufficient class diversity.

2.3 Class Distribution

The dataset was verified for balance.

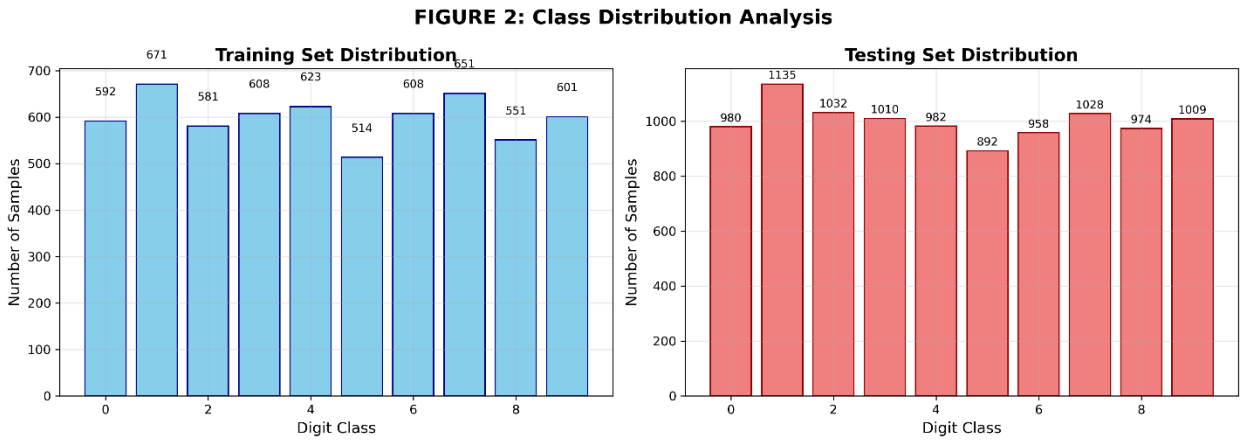
Training distribution (6,000 samples):

- All digits are approximately evenly distributed (~9–11% per class)

Testing distribution (10,000 samples):

- All digits are also evenly distributed

Conclusion:
The dataset is reasonably balanced and suitable for classification tasks.



3. Image Preprocessing

The following preprocessing steps were applied:

1. Normalization
   Pixel values were scaled from [0,255] to [0,1] using:
2. x = x / 255.0
3. Reshaping
   Images were reshaped from:
   (28, 28) → (28, 28, 1)
4. to make them compatible with CNN input format.
5. Label Encoding
   Labels were converted to categorical format using one-hot encoding:
   to_categorical(y, 10)
6. Training–Validation Split
   - 90% training data
   - 10% validation data

4. Baseline Model Design

A Convolutional Neural Network (CNN) was selected because:

- CNNs automatically extract spatial features
- They are highly effective for image classification
- They reduce manual feature engineering

4.1 Model Architecture

The model consists of:

Layer 1:

- Conv2D (32 filters, 3×3, ReLU)
- MaxPooling2D (2×2)
- Dropout (0.25)

Layer 2:

- Conv2D (64 filters, 3×3, ReLU)
- MaxPooling2D (2×2)
- Dropout (0.5)

Layer 3:

- Flatten

Layer 4:

- Dense (128 units, ReLU)
- Dropout (0.5)

Output Layer:

- Dense (10 units, Softmax)

Total parameters: 225,034
Trainable parameters: 225,034

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 26, 26, 32)	320
maxpool_1 (MaxPooling2D)	(None, 13, 13, 32)	0
dropout_1 (Dropout)	(None, 13, 13, 32)	0
conv2d_2 (Conv2D)	(None, 11, 11, 64)	18,496
maxpool_2 (MaxPooling2D)	(None, 5, 5, 64)	0
dropout_2 (Dropout)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense_128 (Dense)	(None, 128)	204,928
dropout_3 (Dropout)	(None, 128)	0
output_layer (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)

5. Model Training

5.1 Training Configuration

- Optimizer: Adam
- Loss Function: Categorical Crossentropy
- Metrics: Accuracy
- Epochs: 10
- Batch Size: 80
- Validation Split: 10%

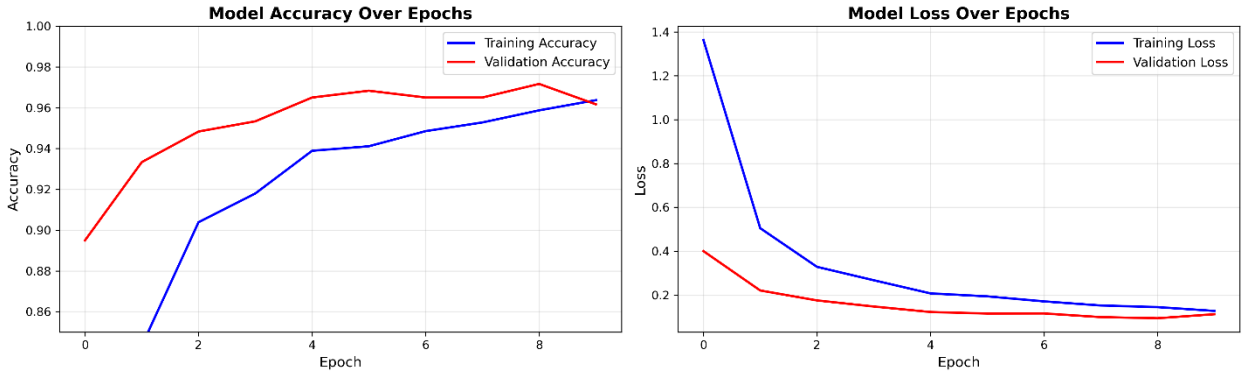
5.2 Training Performance

Final Epoch Results:

- Training Accuracy: 96.37%
- Validation Accuracy: 96.17%
- Training Loss: 0.1275
- Validation Loss: 0.1123

Observation:
Training and validation curves closely follow each other, indicating stable learning.

FIGURE 3: Training and Validation Performance



6. Model Evaluation

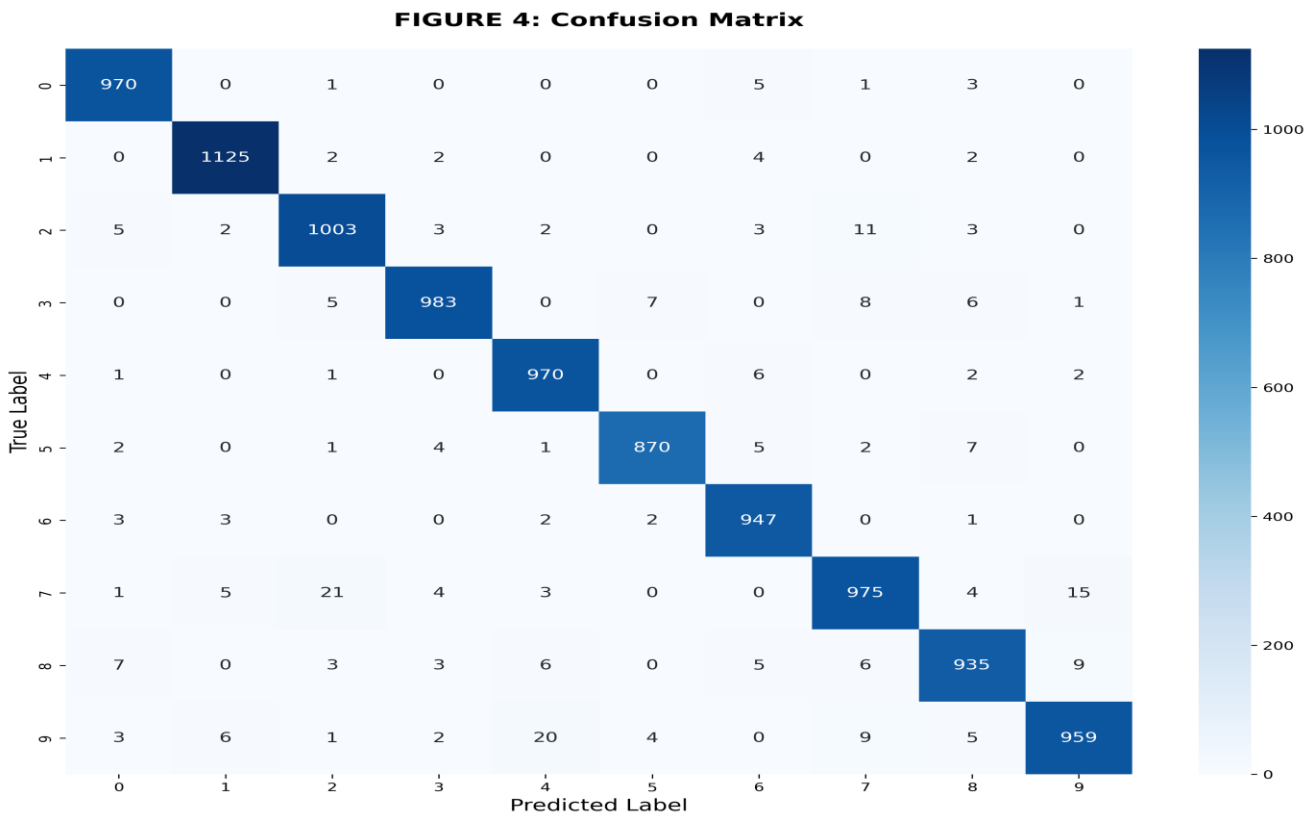
The model was evaluated on the unseen test dataset (10,000 images).

6.1 Test Set Performance

- Test Accuracy: 97.37%
- Test Loss: 0.0806

The model generalizes well despite being trained on only 6,000 samples.

6.2 Confusion Matrix



Key observations:

- Digit 0 achieved highest accuracy (99.08%)
- Digit 9 achieved lowest accuracy (92.96%)
- Common confusion patterns:
    - 7 confused with 2
    - 9 confused with 4
    - 5 confused with 8

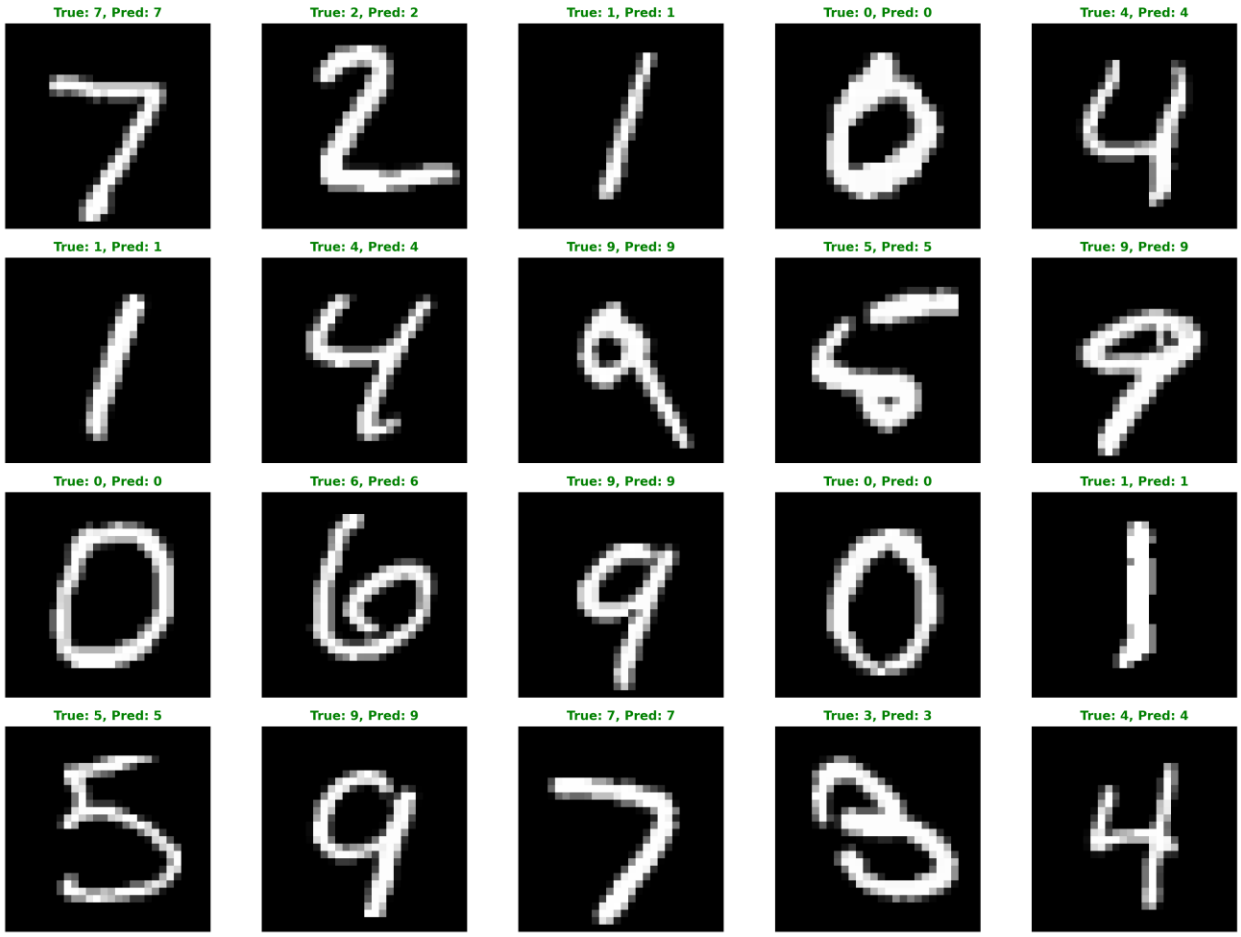
6.3 Classification Report

Overall accuracy: 97%

Precision, Recall, and F1-score are consistently high across all classes.

7. Sample Predictions

FIGURE 5: Sample Predictions (Green=Correct, Red=Incorrect)



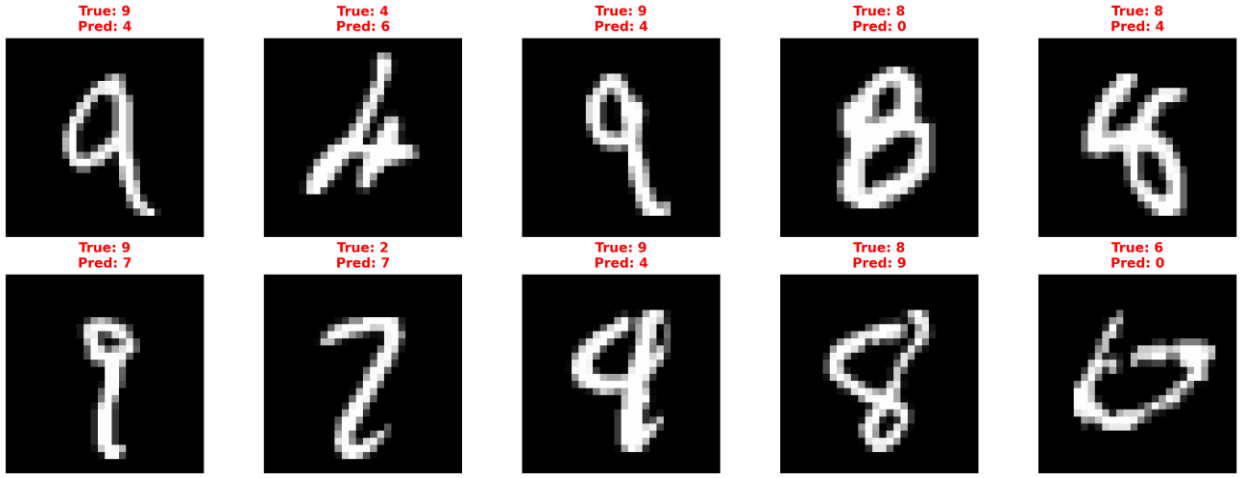
8. Error Analysis

Total misclassified images: 263 (approx.)
Overall accuracy: 97.37%

Common misclassification patterns:

- Digit 7 confused with 2 (21 times)
- Digit 9 confused with 4 (20 times)
- Digit 2 confused with 7 (11 times)

FIGURE 6: Misclassified Examples



These errors are mainly due to similarity in handwritten shapes.

9. Overfitting / Underfitting Analysis

Accuracy Gap (Train – Validation): 0.0020
Loss Gap (Validation – Train): -0.0152

Conclusion:
The model shows minimal overfitting and good generalization capability.

Dropout layers helped prevent overfitting despite reduced training data.

10. Class Imbalance Analysis

Accuracy Range Across Classes:

- Minimum: 92.96% (Digit 9)
- Maximum: 99.08% (Digit 0)
- Variance: 6.12%

Conclusion:
While the dataset itself is balanced, performance variation exists due to shape similarity between certain digits.

11. Model Saving

The trained model was saved in:

saved_models/handwritten_digit_cnn_model.h5

12. Initial Observations

Strengths:

- High accuracy (97.37%)
- Good generalization
- Stable training curves
- Minimal overfitting

Limitations:

- Slight performance variation across digits
- Some confusion between visually similar digits
- Reduced training dataset may limit maximum achievable accuracy

13. Conclusion (Implementation 1)

A baseline CNN-based handwritten digit recognition system was successfully developed. Despite using only 6,000 training samples, the model achieved 97.37% accuracy on the test dataset.

The system demonstrates effective feature extraction, stable training behavior, and strong generalization performance.

Further improvements will be explored in Implementation 2.